Hoja de respuestas y evidencias

FILA A · Sistema de Préstamos de Equipamiento Tecnológico

Nombre: Emily Jara Fecha: 10 de noviembre, 2026

Instrucciones: responda usando su propio programa. Pegue fragmentos breves de código (no archivos completos) y registre los resultados observados. Total: 20 puntos.

Pregunta 1 · 4 puntos

Muestre la definición de Prestamo, sus claves foráneas y relaciones con Equipo y Estudiante. Explique qué relaciona cada clave.

Archivo y función: models.py (Clase Prestamo)

Código / respuesta:

```python
class Prestamo(Base):
    __tablename__ = "prestamos"

    id = Column(Integer, primary_key=True, autoincrement=True)
    equipo_id = Column(Integer, ForeignKey("equipos.id"), nullable=False)
    estudiante_id = Column(
        Integer, ForeignKey("estudiantes.id"), nullable=False
    )
    fecha = Column(
        String, nullable=False, default=lambda: date.today().isoformat()
    )
    cantidad = Column(Integer, nullable=False)
    estado = Column(String, nullable=False, default="ACTIVO")

    equipo = relationship("Equipo", back_populates="prestamos")
    estudiante = relationship("Estudiante", back_populates="prestamos")

    __table_args__ = (
        CheckConstraint("cantidad > 0", name="chk_cantidad_positiva"),
        CheckConstraint(
            "estado IN ('ACTIVO', 'DEVUELTO')", name="chk_estado_valido"
        ),
    )
```

Explicación : La entidad Prestamo relaciona las tablas equipos y estudiantes mediante las claves foráneas equipo_id (referenciando a equipos.id) y estudiante_id (referenciando a estudiantes.id). Las directivas relationship() permiten acceder a los objetos asociados (prestamo.equipo y prestamo.estudiante). Almacena la fecha en formato AAAA-MM-DD, la cantidad prestada y el estado ("ACTIVO" o "DEVUELTO").

Pregunta 2 · 3 puntos

Muestre la función que registra un equipo. Señale dónde utiliza session.add() y cómo confirma la persistencia.

Archivo y función: crud.py (Función crear_equipo)

Código / respuesta:

```python
def crear_equipo(db: Session, nombre: str, tipo: str, stock_disponible: int):
    if stock_disponible < 0:
        raise ValueError("El stock no puede ser negativo.")
    equipo = Equipo(
        nombre=nombre, tipo=tipo, stock_disponible=stock_disponible
    )
    db.add(equipo)
    db.commit()
    db.refresh(equipo)
    return equipo
```

Explicación:: La función invoca db.add(nuevo_equipo) para colocar el nuevo objeto Equipo en la sesión de SQLAlchemy en estado pendiente. La persistencia en la base de datos SQLite se confirma de forma definitiva al ejecutar db.commit(). Finalmente, db.refresh(nuevo_equipo) actualiza la instancia cargando el id autogenerado.

Pregunta 3 · 3 puntos

Copie una consulta ORM con select() que obtenga equipos con stock_disponible mayor que cero. Explique el filtro.

Archivo y función: crud.py

Código / respuesta:

```python
class Equipo(Base):
    __tablename__ = "equipos"

    id = Column(Integer, primary_key=True, autoincrement=True)
    nombre = Column(String, nullable=False)
    tipo = Column(String, nullable=False)
    stock_disponible = Column(Integer, nullable=False, default=0)

    prestamos = relationship("Prestamo", back_populates="equipo")

    __table_args__ = (
        CheckConstraint(
            "stock_disponible >= 0", name="chk_stock_no_negativo"
        ),
    )
```

```python
def crear_equipo(db: Session, nombre: str, tipo: str, stock_disponible: int):
    if stock_disponible < 0:
        raise ValueError("El stock no puede ser negativo.")
    equipo = Equipo(
        nombre=nombre, tipo=tipo, stock_disponible=stock_disponible
    )
    db.add(equipo)
    db.commit()
    db.refresh(equipo)
    return equipo


def listar_equipos(db: Session):
    return db.scalars(select(Equipo)).all()
```

Explicación: Se construye una sentencia ORM con select(Equipo) y se le aplica la condición stock_disponible > 0.. Este filtro evalúa la columna de inventario para recuperar únicamente los equipos que cuentan con al menos una unidad disponible, excluyendo los que están en 0.

Pregunta 4 · 4 puntos

Muestre el código que verifica la disponibilidad antes de registrar un préstamo. ¿Qué ocurre cuando se solicita una cantidad superior al stock_disponible?

Archivo y función: crud.py (Función registrar_prestamo)

Código / respuesta:

```python
if equipo.stock_disponible < cantidad:
    raise ValueError(
        f"Stock insuficiente. Disponibles: {equipo.stock_disponible}, Solicitados: {cantidad}"
    )
```

```
--- SISTEMA DE PRÉSTAMOS DE EQUIPAMIENTO TECNOLÓGICO ---
1. Registrar nuevo equipo
2. Listar equipos
3. Buscar equipo por ID
4. Actualizar equipo
5. Eliminar equipo
6. Registrar estudiante
7. Listar estudiantes
8. Registrar préstamo
9. Procesar devolución
0. Salir
Seleccione una opción: 8
ID del equipo: 1
ID del estudiante: 1
Cantidad a prestar: 6
[ERROR DE VALIDACIÓN] Stock insuficiente. Disponibles: 5, Solicitados: 6
```

Explicación: Al solicitar una cantidad superior al stock disponible, se verifica el stock actual y en caso de ser menor a la cantidad solicitada, se envia un error se validación debido a un stock insuficiente.

Pregunta 5 · 3 puntos

Muestre el código que procesa una devolución. Explique cómo restituye la disponibilidad y cómo impide una devolución duplicada.

Archivo y función: crud.py (Función procesar_devolución)

Código / respuesta:

```python
def procesar_devolucion(db: Session, prestamo_id: int):
    prestamo = db.get(Prestamo, prestamo_id)
    if not prestamo:
        raise ValueError("El préstamo no existe.")
    if prestamo.estado == "DEVUELTO":
        raise ValueError("El préstamo ya fue devuelto previamente.")

    prestamo.estado = "DEVUELTO"
    equipo = db.get(Equipo, prestamo.equipo_id)
    equipo.stock_disponible += prestamo.cantidad
    db.commit()
    db.refresh(prestamo)
    return prestamo
```

Explicación: Para restituir la disponibilidad, el programa lo que hace es sumar al stock disponible del equipo la cantidad devuelta en el prestamo y para impedir una devolución duplicada verifica si el prestamo tiene como estado “DEVUELTO”.

Pregunta 6 · 3 puntos

Registre Notebook UCT con disponibilidad 5; preste 2 unidades y devuélvalas. Escriba el stock observado antes, después del préstamo y después de la devolución, e indique cómo lo verificó.

Momento	Disponibilidad observada
Antes del préstamo	5
Después del préstamo	3
Después de la devolución	5

Forma de verificación:

Antes del prestamo:

```
LISTADO DE EQUIPOS:
ID: 1 | Nombre: Notebook UCT | Tipo: Notebook | Stock Disponible: 5
ID: 2 | Nombre: Proyector | Tipo: Proyector | Stock Disponible: 3
```

Después del prestamo:

```
LISTADO DE EQUIPOS:
ID: 1 | Nombre: Notebook UCT | Tipo: Notebook | Stock Disponible: 3
ID: 2 | Nombre: Proyector | Tipo: Proyector | Stock Disponible: 3
```

Después de la devolución:

```
--- SISTEMA DE PRÉSTAMOS DE EQUIPAMIENTO TECNOLÓGICO ---
1. Registrar nuevo equipo
2. Listar equipos
3. Buscar equipo por ID
4. Actualizar equipo
5. Eliminar equipo
6. Registrar estudiante
7. Listar estudiantes
8. Registrar préstamo
9. Procesar devolución
0. Salir
Seleccione una opción: 9
ID del préstamo a devolver: 2
-> Devolución procesada correctamente.

--- SISTEMA DE PRÉSTAMOS DE EQUIPAMIENTO TECNOLÓGICO ---
1. Registrar nuevo equipo
2. Listar equipos
3. Buscar equipo por ID
4. Actualizar equipo
5. Eliminar equipo
6. Registrar estudiante
7. Listar estudiantes
8. Registrar préstamo
9. Procesar devolución
0. Salir
Seleccione una opción: 2

LISTADO DE EQUIPOS:
ID: 1 | Nombre: Notebook UCT | Tipo: Notebook | Stock Disponible: 5
ID: 2 | Nombre: Proyector | Tipo: Proyector | Stock Disponible: 3
```

Criterios de corrección

Preguntas de 4 puntos: 2 puntos por código pertinente, 1 por explicación técnica y 1 por coherencia con el proyecto. Preguntas de 3 puntos: 1 punto por código/procedimiento, 1 por explicación y 1 por evidencia o resultado. No se aceptan fragmentos que no estén presentes en la entrega.